AI ASSISTANT CODING

Lab Assignment-9.1

2303A54012
A.Sanjay
B47A

Problem 1:

Consider the following Python
function: def find_max(numbers):
return max(numbers)

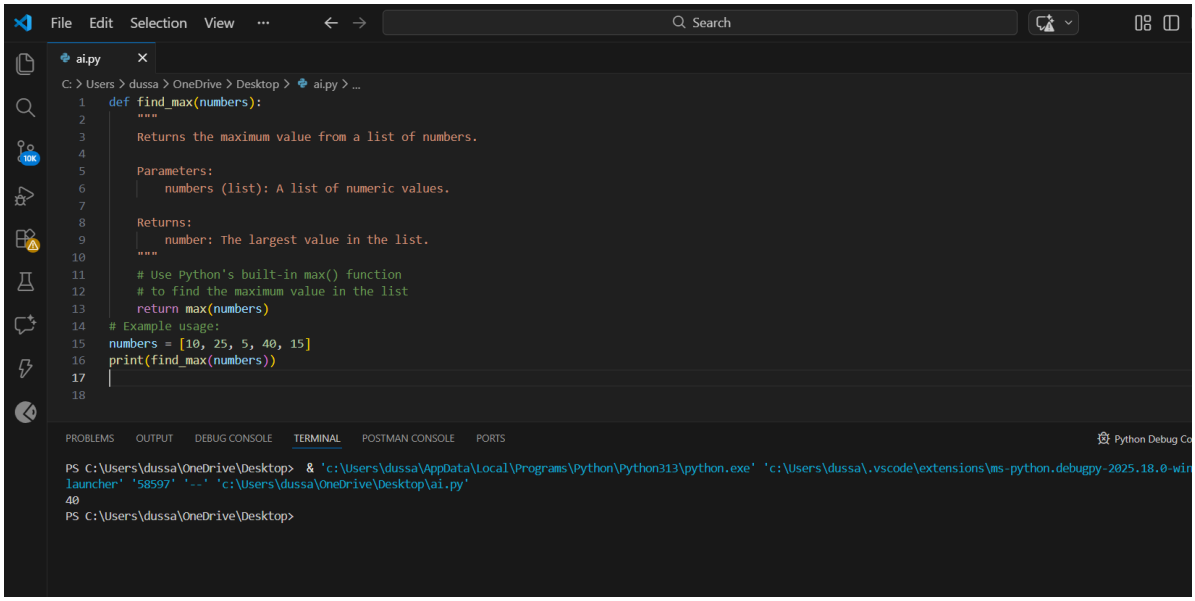
Task:

• Write documentation for the function in all three formats:
(a) Docstring
(b) Inline comments
(c) Google-style documentation
• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

Prompt:

“Generate documentation for the given Python function in three formats: docstring, inline comments, and Google-style documentation. Also compare these documentation styles and recommend the best one for a mathematical utility library.”
give me the code and ouput

Code:



Explanation:

The function find_max() returns the maximum value from a list of numbers. AI is used to automatically generate different documentation styles to understand how documentation improves code clarity and usability.

- Docstring explains the function purpose, parameters, and return value.
- Inline comments describe the logic within the code.
- Google-style documentation provides a structured and professional format suitable for teams and libraries.

Problem 2:

Consider the following Python function:

def login(user, password, credentials):

return credentials.get(user) == password

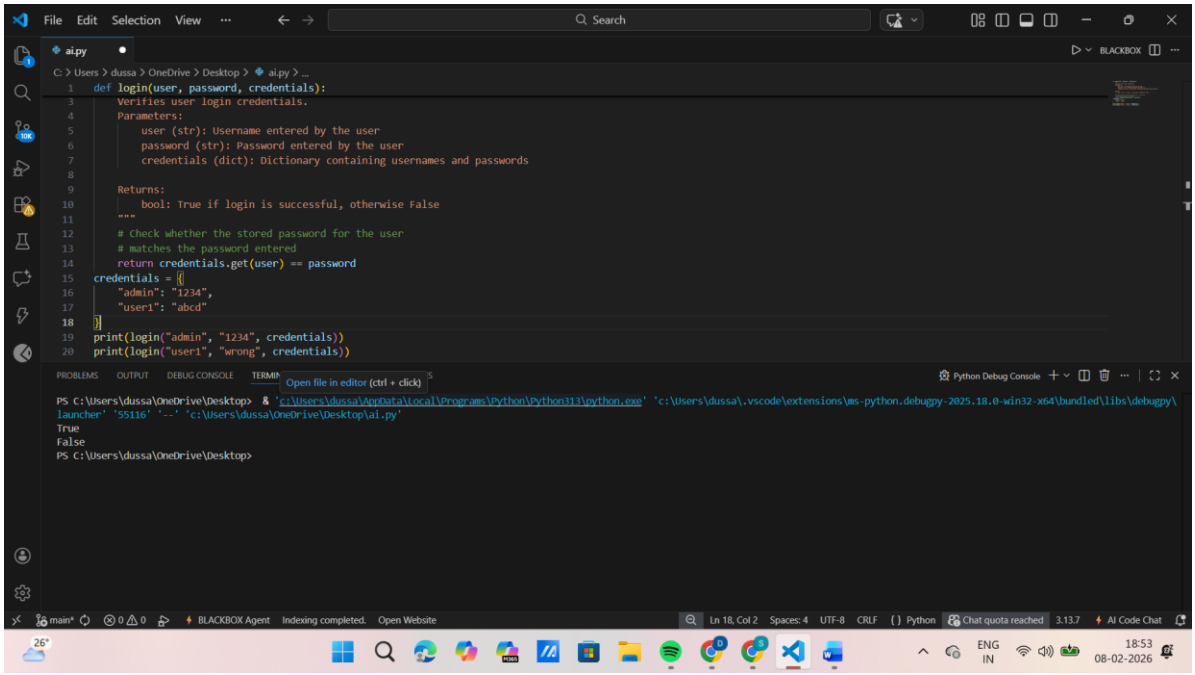
Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Prompt:

“Write docstring, inline comments, and Google-style documentation for the given login function. Compare the approaches and recommend the most helpful style for new developers.”

Code:



Explanation:

The login() function checks whether the entered password matches the stored password for a given user.

Different documentation styles help new developers understand authentication logic quickly and safely.

Google-style documentation is especially helpful during onboarding because it clearly defines parameters and return values.

Problem 3:

Calculator (Automatic Documentation Generation)

Task:

Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:

o add(a, b) – returns the sum of two numbers

o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers

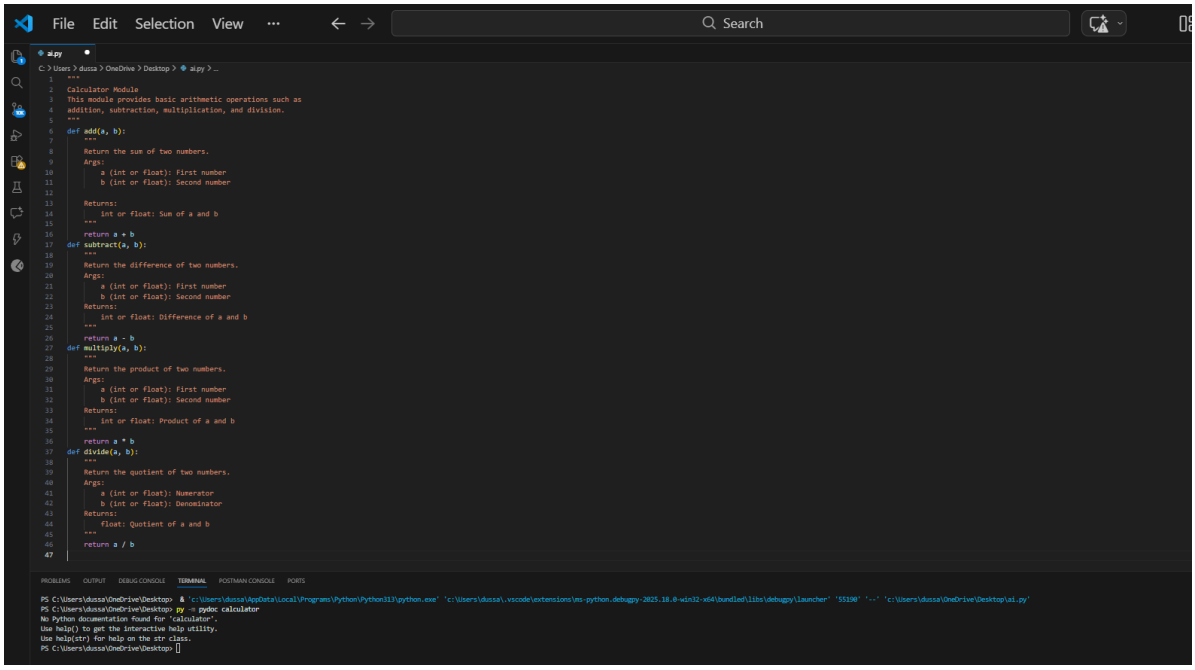
o divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using

Python’s documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

Prompt:
“Create a Python module named calculator.py with arithmetic functions and generate automatic documentation using docstrings and pydoc.”

Code:



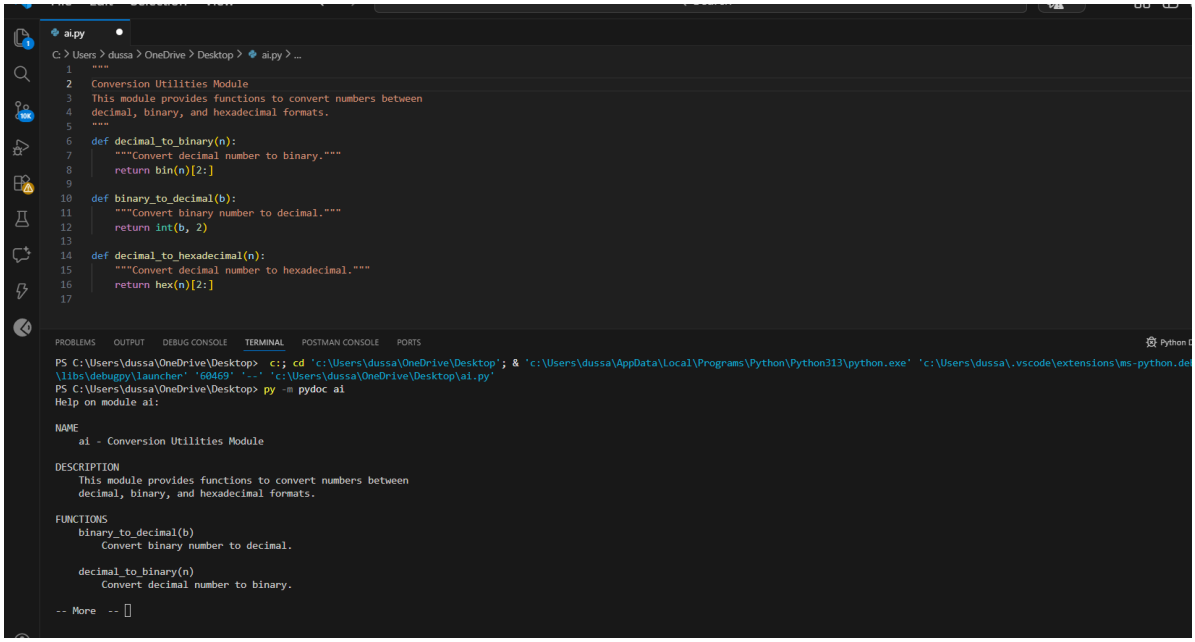
Explanation:
The calculator.py module demonstrates how AI-generated docstrings can be used to automatically create documentation.
Python’s pydoc tool extracts these docstrings and displays them in terminal and HTML format.

Problem 4:
Conversion Utilities Module
Task:
1. Write a module named conversion.py with functions:
o decimal_to_binary(n)
o binary_to_decimal(b)

o decimal_to_hexadecimal(n)
2. Use Copilot for auto-generating docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

Prompt:
“Using Copilot, generate docstrings for number conversion functions and demonstrate automatic documentation generation using pydoc.”

Code:



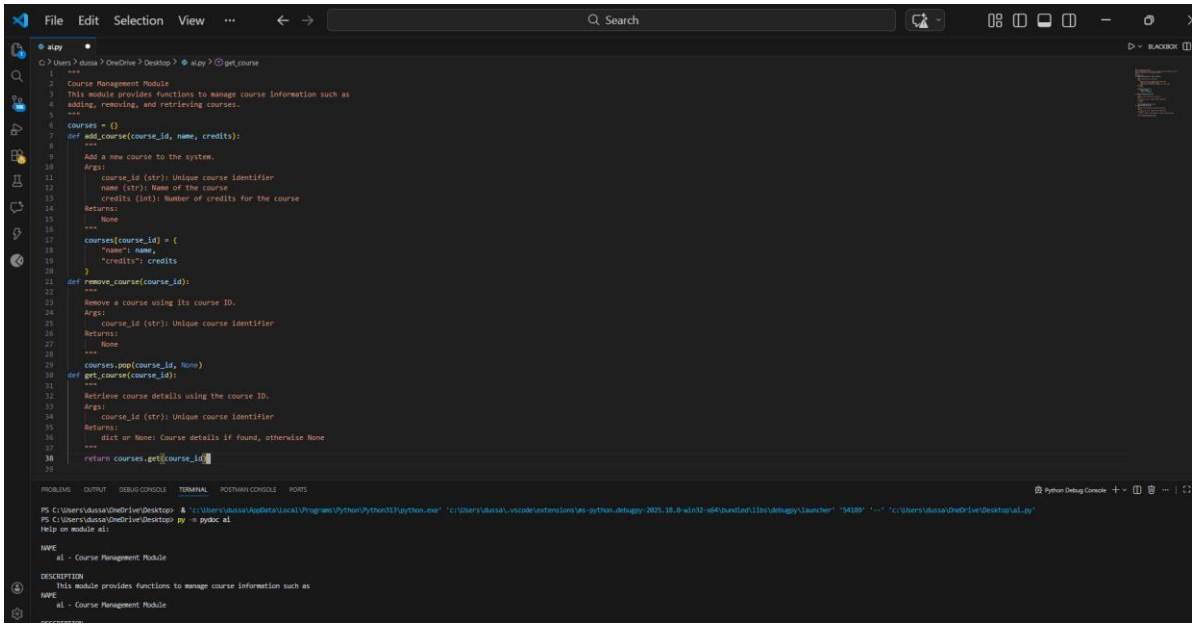
Explanation:
This module converts numbers between decimal, binary, and hexadecimal formats.
AI-generated docstrings ensure consistency and reduce manual documentation effort.

Problem 5 – Course Management Module
Task:
1. Create a module course.py with functions:
o add_course(course_id, name, credits)
o remove_course(course_id)

o get_course(course_id)
2. Add docstrings with Copilot.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format and open it in a browser.

Prompt Used:
“Create a course management module with AI-generated docstrings and demonstrate terminal and HTML documentation generation.”

Code:



Explanation:
This module manages course details using basic CRUD operations.
AI-generated docstrings improve maintainability and help future developers understand module behavior quickly.